

CPSC 250

Inheritance and Polymorphism

Edward Brash

1 Motivation

In many programs, several classes share many of the same features.

If we write those features separately in every class, we duplicate code.

That is inefficient and hard to maintain.

Inheritance gives us a better strategy.

2 The Basic Idea

Inheritance allows us to define a base class containing common information and behavior.

Derived classes then inherit from that base class and add specialized information.

3 Example: Businesses

Suppose we want to model several kinds of businesses.

A general **Business** class might contain:

- name,
- address,
- hours of operation.

A **Restaurant** class might include all of that plus:

- cuisine type,
- rating.

A **CarDealership** class might include all business information plus:

- car makes sold,
- average price by make.

4 Example: Grocery Store Items

Another example is grocery store items.

The base class might be:

Item

with:

- name,
- quantity.

Derived classes might include:

- Produce,
- Cereal,
- Meat.

Each derived class has the base features, plus additional specialized features.

5 Base Class: Item

```
class Item:

    def __init__(self):
        self.name = ""
        self.quantity = 0

    def set_name(self, name):
        self.name = name

    def set_quantity(self, quantity):
        self.quantity = quantity

    def display(self):
        print(self.name, self.quantity)
```

6 Derived Class: Produce

```
class Produce(Item):

    def __init__(self):
        Item.__init__(self)
        self.expiration = ""

    def set_expiration(self, expiration):
        self.expiration = expiration

    def get_expiration(self):
        return self.expiration
```

7 Calling the Base Class Constructor

The line:

```
Item.__init__(self)
```

is an explicit call to the base class constructor.

This initializes the inherited variables:

- `name`,
- `quantity`.

This must be done because Python does not automatically call the base constructor in this style.

8 Main Program Example

```
item1 = Item()
item1.set_name("Cereal")
item1.set_quantity(9)
item1.display()

item2 = Produce()
item2.set_name("Apples")
item2.set_quantity(40)
item2.set_expiration("5/5/2024")
item2.display()

print(f"Expires on {item2.get_expiration()}")
```

9 Why This Is Useful

The derived class `Produce` can use inherited methods from `Item`:

- `set_name()`,
- `set_quantity()`,
- `display()`.

It only needs to define what is new:

- expiration date.

10 Polymorphism

Polymorphism means that different classes can be treated in a common way.

For example, if several objects all have a method called:

```
display()
```

then code can call `display()` without needing to know exactly which type of object it is handling.

11 Inheritance Chains

Inheritance can continue over multiple levels.

For example:

Base → DerivedOne → DerivedTwo

Each class adds information to the previous class.

```
class Base:

    def __init__(self):
        self.name = " "

class DerivedOne(Base):

    def __init__(self):
        Base.__init__(self)
        self.address = " "

class DerivedTwo(DerivedOne):

    def __init__(self):
        DerivedOne.__init__(self)
        self.zip_code = " "
```

12 Important Design Point

The notes emphasize that explicitly calling the base class constructor may feel unusual if you come from other languages.

However, it reflects the spirit of inheritance:

The derived class should intentionally initialize the base-class portion of itself.

13 Key Takeaways

- Inheritance reduces duplicated code.
- A base class stores common features.
- A derived class adds specialized features.
- Derived classes can use inherited methods.
- Python often requires explicit base constructor calls.
- Polymorphism allows different objects to be used through common methods.